

SocialMetrics AI

Rapport d'évaluation

API d'analyse de sentiments pour tweets

Modèle : Régression logistique (scikit-learn)

Client : Daunale Treupe

Surveillance des opinions sur X (Twitter)

Auteur : Corentin Clero

Date : juillet 2026

Projet final — Algorithmes avancés

1. Introduction et contexte

Objectifs et périmètre de l'étude

Contexte

SocialMetrics AI développe pour son client Daunale Treupe un service de surveillance des opinions exprimées sur X (anciennement Twitter).

L'enjeu est de pouvoir évaluer automatiquement, à grande échelle, le sentiment (positif, négatif ou neutre) d'un ensemble de tweets.

Objectifs

- Construire une API Flask exposant un endpoint POST d'analyse.
- Entraîner un modèle de régression logistique sur des tweets annotés stockés dans une base MySQL.
- Automatiser le réentraînement hebdomadaire du modèle (cron).
- Évaluer les performances via matrices de confusion, précision, rappel et F1-score, et identifier les biais du modèle.

Périmètre de ce rapport

Ce document présente l'évaluation chiffrée du modèle entraîné sur le jeu de données annotées disponibles (68 tweets), puis une analyse des forces, faiblesses et biais, suivie de recommandations d'amélioration.

2. Méthodologie

Pipeline d'apprentissage et d'évaluation

Données

- Source : table MySQL tweets (colonnes id, text, positive, negative).
- Annotation : positive=1 → label +1 ; negative=1 → label -1 ; sinon label 0 (neutre).
- Volume : 68 tweets annotés (30 positifs, 30 négatifs, 8 neutres).
- Prétraitement (clean_text) : minuscules, suppression des URL, mentions (@), du # et de la ponctuation.

Modèle

- Vectorisation : TF-IDF (TfidfVectorizer) sur les textes nettoyés.
- Classifieur : LogisticRegression (scikit-learn, max_iter=1000).
- Sortie 3 classes : -1 (négatif), 0 (neutre), 1 (positif).
- Score API : $\text{predict_proba}(+1) - \text{predict_proba}(-1) \in [-1 ; 1]$.

Évaluation

- Découpage train/test : 75 % / 25 %, stratifié, random_state=42.
- Jeu de test : 17 tweets (8 négatifs, 7 positifs, 2 neutres).
- Métriques : classification_report (précision, rappel, F1) + matrices de confusion binaires (positif vs reste, négatif vs reste).

3. Résultats — performances globales

Régression logistique 3 classes (-1, 0, +1)

Rapport de classification (jeu de test, 17 tweets)

-1 (négatif)	0.73	1.00	0.84	8
0 (neutre)	0.00	0.00	0.00	2
+1 (positif)	0.67	0.57	0.62	7
accuracy			0.71	17
macro avg	0.46	0.52	0.49	17
weighted avg	0.62	0.71	0.65	17

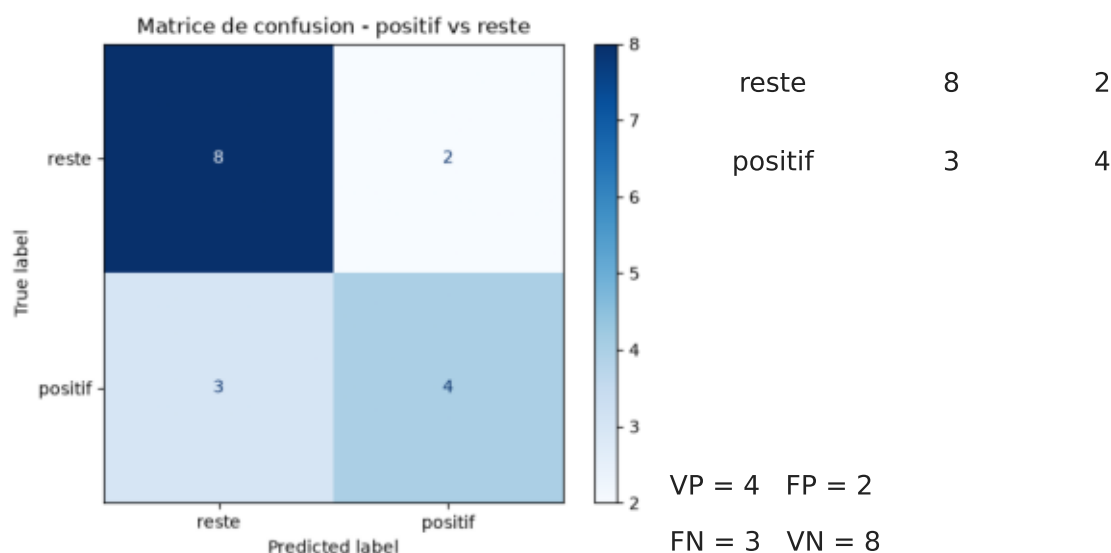
Matrice de confusion globale (vraie \ prédite)

vraie -1	8	0	0
vraie 0	0	0	2
vraie +1	3	0	4

Lecture : aucun négatif n'est manqué (colonne -1 = 8+0+3). En revanche, la classe neutre (0) n'est jamais prédite : ses 2 exemplaires sont classés en positif (+1). Trois positifs sont à tort classés négatifs.

4. Matrice de confusion — classe positive

Binaire : positif vs reste



Métriques (classe positive)

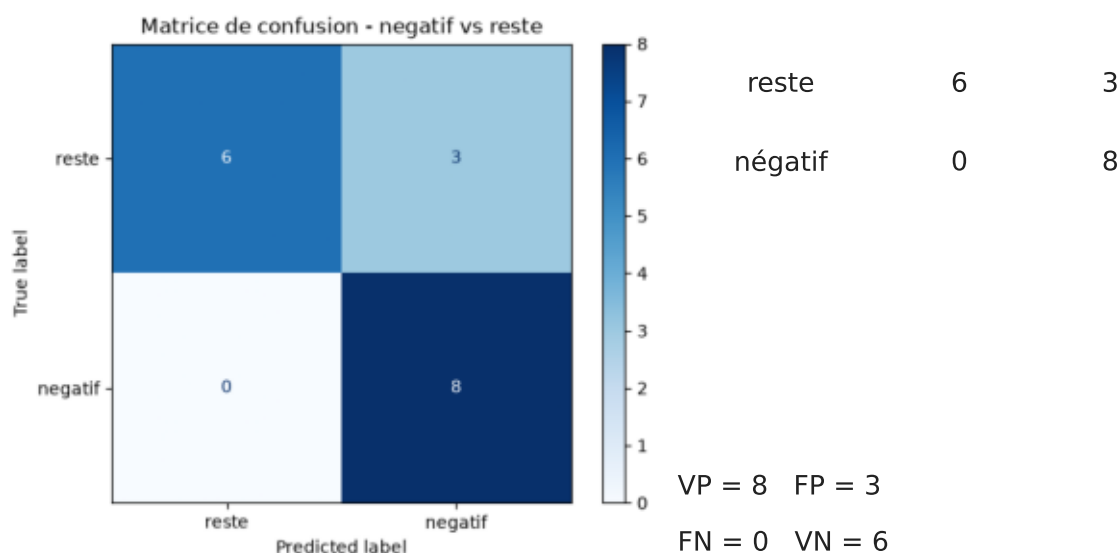
- Précision = 0.67 • Rappel = 0.57 • F1 = 0.62
- Accuracy (positif vs reste) = 0.71

Interprétation

Sur 7 vrais positifs, 4 sont correctement détectés (rappel 57 %) ; 3 sont à tort classés « reste » (faux négatifs). La précision 0.67 indique que sur 6 prédictions « positif », 2 sont en réalité des restes (faux positifs), souvent des tweets neutres. Le modèle a donc une tendance à sous-détecter les tweets positifs.

5. Matrice de confusion — classe négative

Binaire : négatif vs reste



Métriques (classe négative)

- Précision = 0.73 • Rappel = 1.00 • F1 = 0.84
- Accuracy (négatif vs reste) = 0.82

Interprétation

La détection des négatifs est excellente : rappel = 1.00, aucun vrai négatif manqué (FN = 0). C'est la classe la mieux identifiée. En revanche, 3 « restes » (en fait des positifs) sont classés négatifs (FP = 3), d'où une précision plus modeste (0.73). Pour la surveillance de marque, ce biais « prudent » est acceptable : mieux vaut signaler un faux négatif que rater une critique réelle.

6. Analyse des performances

Forces, faiblesses et biais identifiés

Forces

- Détection des tweets négatifs très fiable (rappel 1.00, F1 0.84).
- Pipeline reproductible (random_state fixé) et automatisable.
- Architecture en API + cron de réentraînement opérationnelle.

Faiblesses

- Classe neutre jamais prédite (précision/rappel/F1 = 0.00).
- Sous-détection des positifs (rappel 0.57 vs 1.00 pour les négatifs).
- Asymétrie de comportement entre positif et négatif.
- Jeu de test très petit (17 tweets) : métriques à forte variance.

Biais

- Biais vers la négativité : le modèle attrape tous les négatifs au prix de 3 positifs classés négatifs (sur-généralisation négative).
- Biais structurel dû au déséquilibre : 30 négatifs / 30 positifs / seulement 8 neutres → la classe minoritaire est ignorée.
- Biais lexical : TF-IDF sur textes courts sans lemmatisation ni stop-words ; les négations et la ironie ne sont pas modélisées.

7. Recommandations

Pistes d'amélioration du modèle

Données

1. Augmenter massivement le jeu annoté (plusieurs milliers de tweets).
2. Rééquilibrer la classe neutre ou utiliser `class_weight='balanced'`.
3. Diversifier les sources pour couvrir sarcasme, argot, emojis.

Modèle

4. Ajouter n-grammes (1,2) et stop-words au vectoriseur TF-IDF.
5. Prétraitement enrichi : lemmatisation/stemming, gestion des négations.
6. Détecter les neutres par seuil sur `predict_proba` ($\max < 0.4 \rightarrow$ neutre).
7. Remplacer la régression logistique par un modèle à représentations denses (word2vec, FastText) ou un transformeur (BERT/CamemBERT).

Évaluation & suivi

8. Validation croisée et métriques par classe à chaque réentraînement.
9. Suivi de la dérive des données via le cron hebdomadaire (comparer les distributions de labels et les métriques d'une semaine à l'autre).
10. Mettre en place un jeu de validation figé pour mesurer les régressions entre réentraînements.